Design Pattern: Panoramica e Iterator

Molka Gharbaoui

Introduzione

Introduzione

- La fase di design di un sistema è una fase cruciale che si interpone tra la fase di analisi e quella di implementazione.
- Un buon design risolve un problema:
  - Definendo al meglio le relazioni tra classi e le gerarchie di ereditarietà
  - Con la possibilità di essere facilmente modificato e/o riusato
  - Con un numero di linee di codice ridotto
  - Con meno uso di risorse

Introduzione

- Per risolvere un problema bisogna identificare soluzioni e strutture simili, ovvero ***patterns***.
- Un pattern serve a risolvere un problema ricorrente documentando una soluzione provata con successo diverse volte.

Introduzione

- Un ***Pattern*** è una descrizione di pratiche di buon senso.
- Una soluzione ad un problema può essere analizzata, estrapolata dal contesto per renderla astratta e più facilmente applicabile in altri domini.
- La descrizione del pattern diventa sufficientemente generica perché rappresenti la soluzione a una famiglia di problemi.

Design Patterns

Design Patterns

- Un ***Design Pattern*** è l'applicazione del concetto di pattern nel dominio dello sviluppo software.
- "Design Patterns, Elements of Reusable Object-Oriented Software", Gamma, Helm, Johnson, Vlissides (detti GOF = Gang Of Four): primi autori ad applicare il concetto di pattern al design a oggetti.

Design Patterns

- Ogni Design Pattern è caratterizzato dai seguenti elementi:
  - ***Nome***: deve essere significativo. Semplifica la comunicazione tra i designers.
  - ***Problema***: un pattern serve per risolvere un problema.
  - ***Soluzione***: elementi partecipanti e le forme di collaborazione e interazione tra questi. Deve essere sufficientemente astratta per applicarla in diversi casi.
  - ***Conseguenze***: un pattern è a volte più adatto di un altro. Serve identificare le sue conseguenze per determinare la sua efficacia.

- ***Applicabilità***: descrive le situazioni in cui il pattern può essere applicato.
- ***Implementazioni***: sono le tecniche e suggerimenti utili all'implementazione del pattern.
- ***Usi conosciuti:*** esempi di applicazione in sistemi reali
- ***Pattern correlati:*** altri pattern

Design Patterns

- I Design Patterns servono a:
  - Introdurre un vocabolario comune
  - capire più facilmente il funzionamento dei sistemi a oggetti
  - Accelerare la fase di apprendimento
  - Anticipare i cambiamenti.
  - Imparare dal lavoro degli altri (evitando errori comuni…)

Design Patterns

- Ci sono diverse categorie di Design Patterns:
  - Pattern di creazione;
  - Pattern di struttura;
  - Pattern di comportamento

Design Patterns

- Pattern di creazione:
  - Astraggono il meccanismo di generazione di istanze di una classe con l’obiettivo di rendere il sistema indipendente da come i suoi oggetti sono creati, composti, e rappresentati.
  - La creazione degli oggetti viene tipicamente gestita da apposite “strutture”
  - forniscono molta flessibilità nel cosa, il come, il chi ed il quando della creazione di oggetti.
  - E.g., Abstract Factory, Singleton, Builder

Design Patterns

- Pattern di struttura:
  - descrivono come comporre oggetti o classi per creare strutture complesse.
  - Si basano sui concetti Object Oriented di ereditarietà e polimorfismo.
  - E.g., Facade, Container, Decorator.

Design Patterns

- Pattern di comportamento:
  - Legati agli algoritmi e gli assegnamenti di responsabilità tra gli oggetti.
  - Caratterizzano sistemi di controllo complessi.
  - permettono di modellare il comportamento del sistema definendo le responsabilità delle sue componenti e definendo le modalità di interazione tra di loro.
  - E.g., Chain of Responsibility, Template Method, Iterator, Mediator, State.

Design Patterns

- Un altro criterio di classificazione riguarda il dominio dei Design Patterns:
  - **Class Pattern**: pattern che definiscono le relazioni fra classi e sottoclassi. Le relazioni sono basate prevalentemente sui concetti di ereditarietà e sono quindi statiche (definite a tempo di compilazione).
  - **Object Pattern**: pattern che definiscono relazioni tra oggetti, che possono cambiare durante l'esecuzione e sono quindi più dinamiche.

Design Patterns

- In totale, ci sono 23 Design Patterns:

1. Adapter
2. Façade
3. Composite
4. Decorator
5. Bridge
6. Singleton
7. Proxy
8. Flyweight
9. Strategy
10. State
11. Command
12. Observer
13. Memento
14. Interpreter
15. Iterator
16. Visitor
17. Mediator
18. Template Method
19. Chain of Responsability
20. Builder
21. Prototype
22. Factory Method
23. Abstract Factory

Design Patterns

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight Observer State Strategy Visitor

Pattern Iterator

Pattern Iterator

- Problema:
  - Molte strutture dati rappresentano aggregati di oggetti (e.g., collezioni, contenitori)
  - I meccanismi di accesso efficienti per una struttura possono non essere efficienti per un’altra
  - E necessario consentire al client di un aggregato di accedere all’elenco degli oggetti senza dipendere dalla struttura

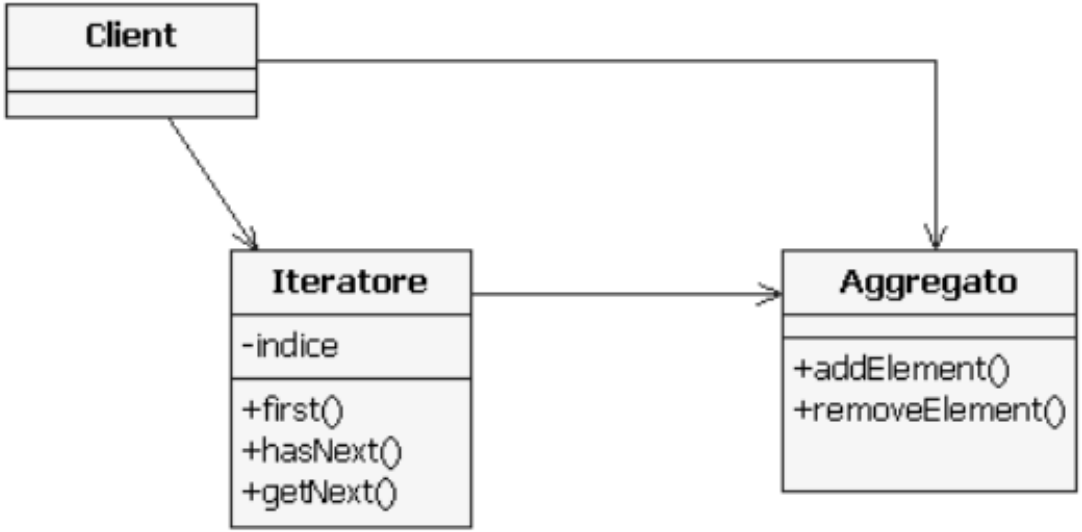
Pattern Iterator

- Contesto:
  - Un oggetto (aggregato) contiene altri oggetti (elementi)
  - I clienti devono accedere a tutti gli elementi
  - L'aggregato non deve esporre la sua struttura interna
  - Più clienti devono poter accedere contemporaneamente
- Soluzione:
  - Definire una classe iteratore che recupera un elemento per volta
  - L'aggregato ha un metodo che restituisce un iteratore

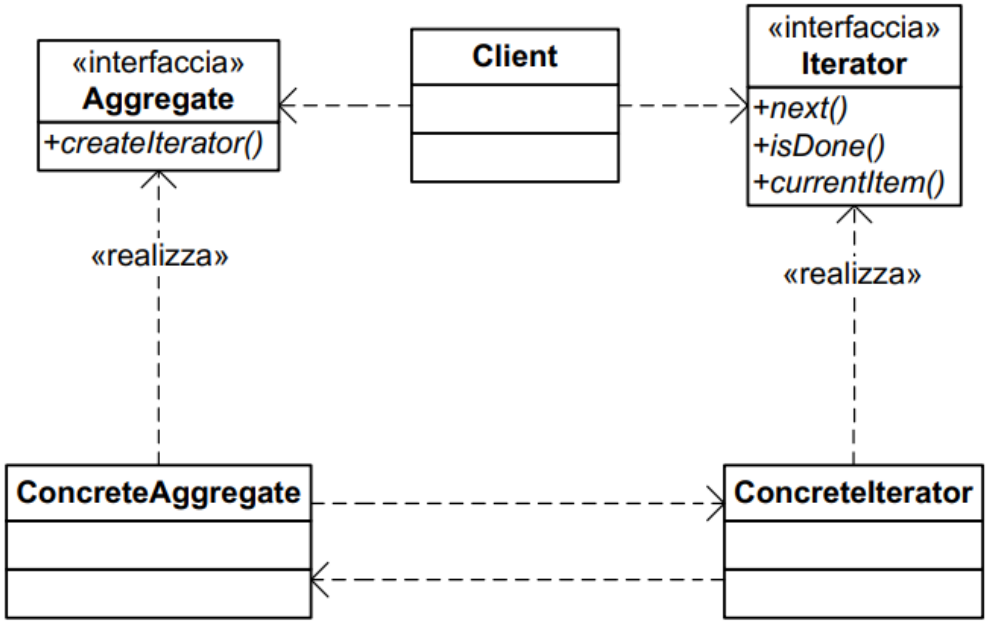
Pattern Iterator

- Il Pattern fornisce un modo per accedere agli elementi di un oggetto aggregato in maniera sequenziale.
- Un oggetto iterator tiene traccia dell'elemento corrente:
  - Conosce quali elementi sono già stati attraversati
  - Consente l'uso contemporaneo di più iteratori sulla stessa struttura dati

Pattern Iterator



Pattern Iterator



Pattern Iterator

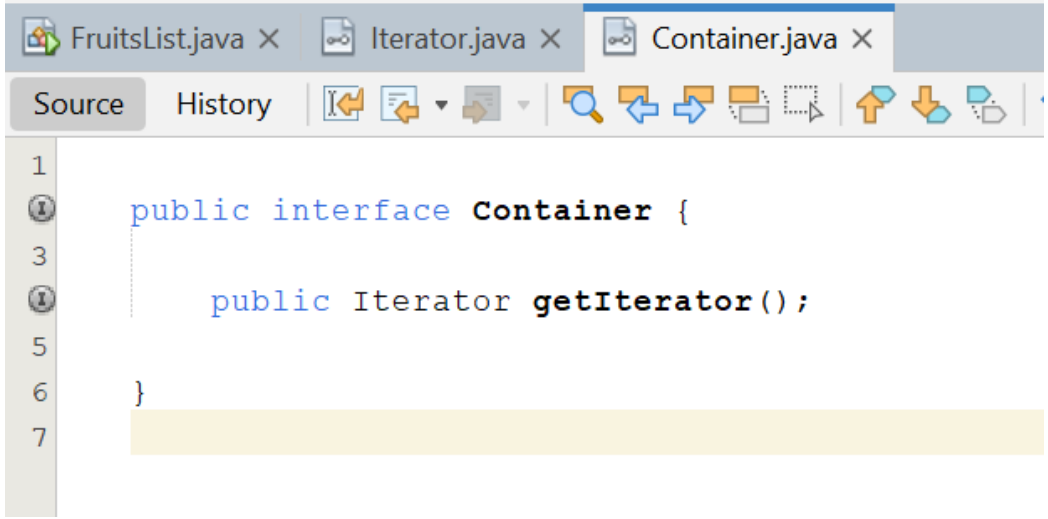
- In Java un *Aggregatore* si ottiene tramite un array, una lista, un set, un'hash, una coda, etc.
- Ogni oggetto aggregatore dispone di propri metodi di accesso e manipolazione dei dati.
- Nelle librerie java questo pattern è già presente attraverso le interfacce:
  - **java.util.Iterator (Iteratore)** : nasconde le strutture dati esponendo sempre gli stessi metodi di interrogazione.
  - **java.lang.Iterable (Aggregatore)**: dichiara il supporto alla funzionalità di iterazione tale da consentire la creazione di un oggetto *Iteratore*

Esempio Implementazione Iterator

Esempio Implementazione Iterator

- Creare una classe ListaFrutta che implementa il pattern Iterator per recuperare i nomi della frutta in un array.

Esempio Implementazione Iterator



Esempio Implementazione Iterator

FruitsList.java | Iterator.java | Container.java

Source | History

```
1
    public interface Iterator {
3
     public boolean hasNext();
     public Object next();
6
    }
8
```

Esempio Implementazione Iterator

FruitsList.java | Iterator.java | Container.java

Source | History

```
1  public class FruitsList implements Container{
2
3  public String fruits[] = {"Banana", "Apple", "Pear", "Peach", "Blueberry"};
4
5  @Override
6  public Iterator getIterator() {
7
8     return new FruitIterator();
9  }
10
11 private class FruitIterator implements Iterator {
12
13 int index;
14
15 @Override
16 public boolean hasNext() {
17
18 return index < fruits.length;
19 }
20
21 @Override
22 public Object next() {
23 if(this.hasNext()) {
24
25 return fruits[index++];
26 }
27 return null;}
28 }
29
```

Esempio Implementazione Iterator

```
29
30  public static void main(String[] args){
31
32   FruitsList listaFrutta = new FruitsList();
33
34   System.out.println( x: "[MAIN]: Recupero lista della frutta");
35
36   for(Iterator iterator=listaFrutta.getIterator(); iterator.hasNext();){
37
38     String frutta = (String)iterator.next();
39     System.out.println("[MAIN]: Frutta -> "+frutta);
40   }
41
42   }
43  }
44
```

Esempio Implementazione Iterator

